Scalability & Future Plan

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	1 Mark

Current System Limitations

S. No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Trained on a simulated, moderate-size dataset rather than real large-scale flood records	Accuracy may not generalize to real-world conditions	High
2	Single-process Flask development server	Limited concurrent request handling	Medium
3	No persistent storage of past predictions	Can't analyze historical usage trends or retrain on real feedback	Medium
4	No authentication or role separation (resident vs. official)	Cannot tailor guidance or restrict admin features	Low

Scalability Plan

S. No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Flask dev server, single process	Deploy with gunicorn + multiple workers behind a reverse proxy (e.g., on IBM Cloud Code Engine)
2	Data Storage	Flat CSV file, local model artifacts	Move to a managed database and object storage for datasets/model versions
3	Performance	In-memory model, sub-second inference	Add response caching and horizontal scaling for peak monsoon-season traffic
4	Security	No authentication, local only	Add HTTPS, rate limiting, and role-based access when exposed publicly

Future Roadmap

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Integrate a real, larger flood dataset (e.g., from Kaggle or government hydrology sources)	Q3 2026	More reliable, generalizable predictions

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 3	Public deployment on IBM Cloud with a production WSGI server	Q4 2026	Accessible to real disaster-management teams
Phase 4	Add historical prediction logging and a simple analytics dashboard	Q1 2027	Enables trend analysis and model retraining feedback loop